Rust Programming Course

A practical guide to ownership, systems programming, and RAG workflows

Language: Rust

How to use this guide

Use this document as a learning reference and as a knowledge source for a Retrieval-Augmented Generation (RAG) experiment. Run the examples in the relevant language environment. Later sections cover document ingestion, chunk metadata, retrieval evaluation, and safety.

Contents

1. Rust Overview
2. Bindings and Mutability
3. Types and Tuples
4. Functions and Control Flow
5. Ownership
6. Borrowing
7. Structs and Enums
8. Option and Result
9. Collections
10. Cargo and Crates
11. Traits and Generics
12. Iterators and Closures
13. Error Propagation
14. File Processing
15. Threads and Shared State
16. Service Design
17. Retrieval-Augmented Generation
18. Chunk Identity and Provenance
19. Security and Evaluation
20. Exercises
21. Summary

1. Rust Overview

Rust is a compiled systems language focused on performance and memory safety. It is used for command-line tools, infrastructure, embedded software, WebAssembly, and performance-sensitive services. Ownership and the type system are central to Rust design.

Example

```
fn main() {
    let component = "retrieval-worker";
    let batch_size = 64;
    println!("{} processes batches of {}", component, batch_size);
}
```

2. Bindings and Mutability

Bindings are immutable by default. Use mut when reassignment or mutation is required. Constants use const and require a type. Shadowing creates a new binding with the same name, possibly with a different type.

Example

```
fn main() {
    let source = "guide.md";
    let mut processed = 0;
    processed += 1;
    println!("{} {}", source, processed);
}
```

3. Types and Tuples

Rust scalar types include integers, floats, booleans, and characters. Tuples group values that may have different types; arrays store a fixed number of values of one type.

Example

```
fn main() {
    let response: (u16, bool) = (200, true);
    let weights: [f32; 3] = [0.2, 0.3, 0.5];
    println!("{} {}", response.0, weights[1]);
}
```

4. Functions and Control Flow

Functions declare parameter and return types. A final expression without a semicolon becomes the return value. Rust provides if expressions, loop, while, and for constructs.

Example

```
fn classify(size: usize) -> &'static str {
    if size == 0 { "empty" }
    else if size < 1024 { "small" }
    else { "large" }
}
fn main() { println!("{}", classify(512)); }
```

5. Ownership

Each value has an owner, and the value is dropped when its owner leaves scope. Moving a value transfers ownership; the previous binding generally cannot be used afterward. These rules help prevent memory errors.

Example

```
fn main() {
```

```
    let original = String::from("source text");
    let moved = original;
    println!("{}", moved);
    // original cannot be used after the move.
}
```

6. Borrowing

Borrowing lets a function access a value without taking ownership. Shared references read values; mutable references allow mutation under strict rules. Rust prevents many conflicting accesses at compile time.

Example

```
fn count_chars(text: &str) -> usize {
    text.chars().count()
}
fn main() {
    let title = String::from("Rust guide");
    println!("{}", count_chars(&title));
}
```

7. Structs and Enums

Structs group fields, while enums represent a value from a defined set of variants. Pattern matching can require every variant to be handled, making state transitions explicit.

Example

```
enum State { Pending, Running, Finished }
struct Job { id: u64, state: State }

fn main() {
    let job = Job { id: 7, state: State::Pending };
    match job.state {
        State::Pending => println!("job {} pending", job.id),
        State::Running => println!("running"),
        State::Finished => println!("finished"),
    }
}
```

8. Option and Result

Option models a value that may be absent. Result models success or failure. Both types encourage explicit handling of missing data and errors rather than unchecked assumptions.

Example

```
fn parse_limit(input: &str) -> Result<usize, std::num::ParseIntError> {
    input.parse::<usize>()
}
fn main() {
    match parse_limit("25") {
        Ok(limit) => println!("limit: {}", limit),
        Err(error) => eprintln!("invalid limit: {}", error),
    }
}
```

9. Collections

Vec stores growable sequences, HashMap supports key-value lookup, and HashSet stores unique values. Choose a collection based on lookup patterns and required behavior.

Example

```
use std::collections::HashMap;
```

```
fn main() {
    let mut counts = HashMap::new();
    counts.insert("documents", 14);
    counts.insert("segments", 96);
    println!("{:?}", counts.get("segments"));
}
```

10. Cargo and Crates

A crate is a Rust compilation unit. Binary crates produce executables, while library crates expose reusable code. Cargo manages dependencies, builds, tests, and package metadata.

Example

```
# Common commands
cargo new retrieval_cli
cd retrieval_cli
cargo check
cargo test
cargo run
```

11. Traits and Generics

Traits define behavior that types can implement. Generics allow functions and data types to work with multiple types while retaining compile-time checks. Trait bounds constrain which behavior a generic type must provide.

Example

```
trait Describe {
    fn describe(&self) -> String;
}
struct Source { name: String }
impl Describe for Source {
    fn describe(&self) -> String {
        format!("source: {}", self.name)
    }
}
```

12. Iterators and Closures

Iterators process sequences lazily. Methods such as filter, map, and collect express transformations without manual indexing. Closures are anonymous functions that can capture values from their environment.

Example

```
fn main() {
    let lengths: Vec<usize> = ["intro", "ownership", "retrieval"]
        .iter()
        .filter(|word| word.len() > 5)
        .map(|word| word.len())
        .collect();
    println!("{:?}", lengths);
}
```

13. Error Propagation

The question-mark operator propagates errors from functions returning compatible Result or Option types. Add context at application boundaries so failures can be diagnosed without exposing sensitive information.

Example

```
use std::{fs, io};

fn read_source(path: &str) -> io::Result<String> {
    fs::read_to_string(path)
```

```
}
fn main() {
    match read_source("README.md") {
        Ok(text) => println!("read {} bytes", text.len()),
        Err(error) => eprintln!("could not read source: {}", error),
    }
}
```

14. File Processing

The standard library supports filesystem access and path manipulation. For large sources, consider streaming rather than loading everything into memory. Validate encoding, size, and structure for untrusted files.

Example

```
use std::fs;

fn main() -> std::io::Result<()> {
    let path = "sample.txt";
    fs::write(path, "Knowledge sources need provenance.")?;
    let contents = fs::read_to_string(path)?;
    println!("{}", contents);
    Ok(())
}
```

15. Threads and Shared State

Rust's safe type system helps prevent data races. Threads run work concurrently; channels transfer messages. Arc and Mutex can coordinate shared state when necessary, but simple message passing is often easier to reason about.

Example

```
use std::thread;
fn main() {
    let worker = thread::spawn(|| "index preparation complete");
    println!("{}", worker.join().expect("worker panicked"));
}
```

16. Service Design

Rust web services can use frameworks such as Axum or Actix Web. Separate transport concerns from business logic, validate requests, and map errors to appropriate status codes. Select a framework based on operational requirements and team expertise.

Example

```
Conceptual endpoints:
GET  /health       -> service status
POST /sources      -> validate and enqueue ingestion
GET  /jobs/{id}    -> return processing status
```

17. Retrieval-Augmented Generation

RAG combines search over external knowledge with language-model generation. A typical workflow extracts source content, creates chunks, computes embeddings, indexes them, retrieves relevant passages, and supplies those passages as model context. RAG still requires evaluation because retrieval alone does not guarantee factual answers.

Example

```
knowledge files
  -> parsing
  -> chunk boundaries and metadata
  -> embedding generation
```

```
  -> vector or hybrid index
  -> query retrieval
  -> context assembly
  -> language-model response
```

18. Chunk Identity and Provenance

Each searchable chunk should be traceable to its source. Store a stable identifier, source path or URI, heading, revision, and relevant permission attributes. Re-index changed material and remove obsolete chunks to avoid stale answers. Heading-aware chunking can preserve meaning better than arbitrary cuts.

Example

```
struct ChunkMetadata {
    id: String,
    source: String,
    heading: String,
    revision: u32,
}
```

19. Security and Evaluation

Retrieved passages are untrusted input and may contain malicious instructions. Keep trusted prompts separate from retrieved text, and apply permission filters before returning passages. Test retrieval against known relevant sources and assess answer grounding, completeness, citations, latency, and cost.

Example

```
Review checklist:
- Maintain a representative query set.
- Inspect retrieved passages and metadata.
- Verify claims against source evidence.
- Enforce access control before returning results.
- Monitor latency and resource use.
```

20. Exercises

1. Write a function accepting &str and returning a character count.
2. Model ingestion status with an enum.
3. Parse a configuration number using Result.
4. Count terms with a HashMap.
5. Read a file and handle filesystem errors.
6. Explain moving a String versus borrowing &str.
7. Design metadata for a versioned handbook.
8. Compare fixed-size and heading-aware chunking.
9. Create a retrieval relevance test set.
10. Explain permission filtering in a RAG application.

21. Summary

Rust offers performance, explicit ownership, expressive types, and strong compile-time checks. Its collections, modules, error handling, and concurrency model are useful for reliable systems software. This course is a separate RAG knowledge source that can be ingested, split, indexed, and queried; keep provenance and access controls with every chunk.